

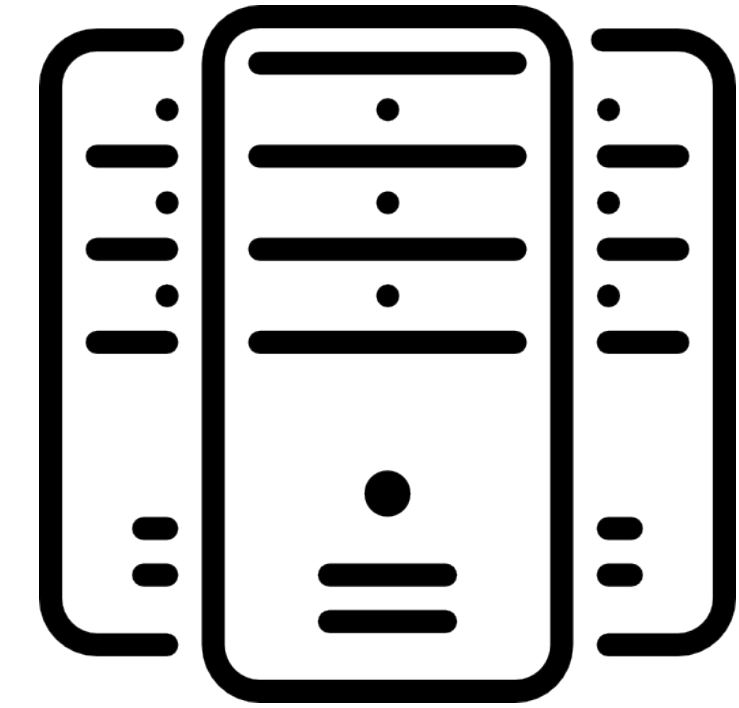
Password Security

Password Security



ALICE

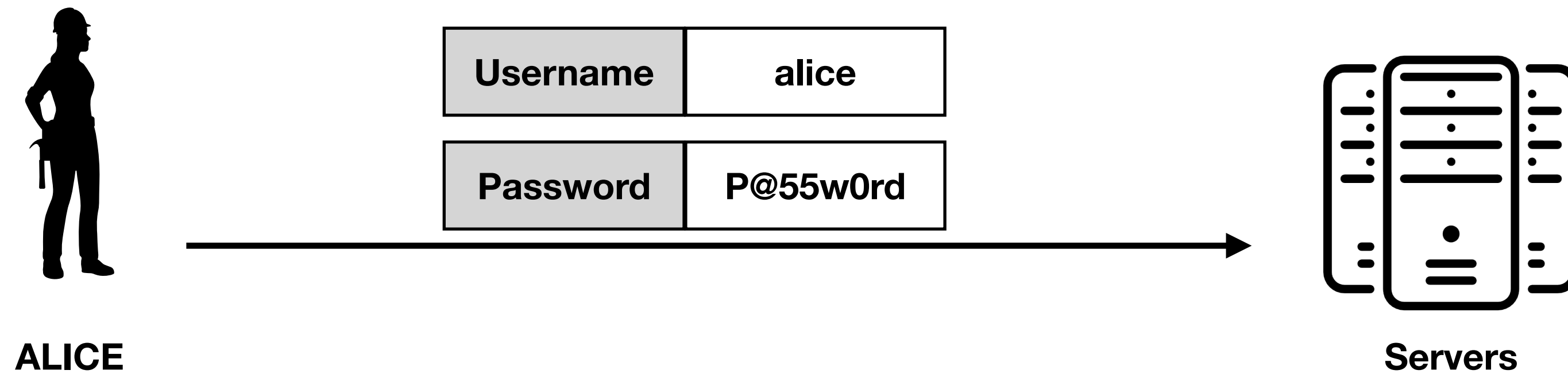
Username	alice
Password	P@55w0rd



Servers

When Alice wants to access services using her username and ***password***, the server needs to ***verify*** her identity. This depends on the information that the server already holds about Alice. First, the server needs to verify her username, then her password.

Password Security



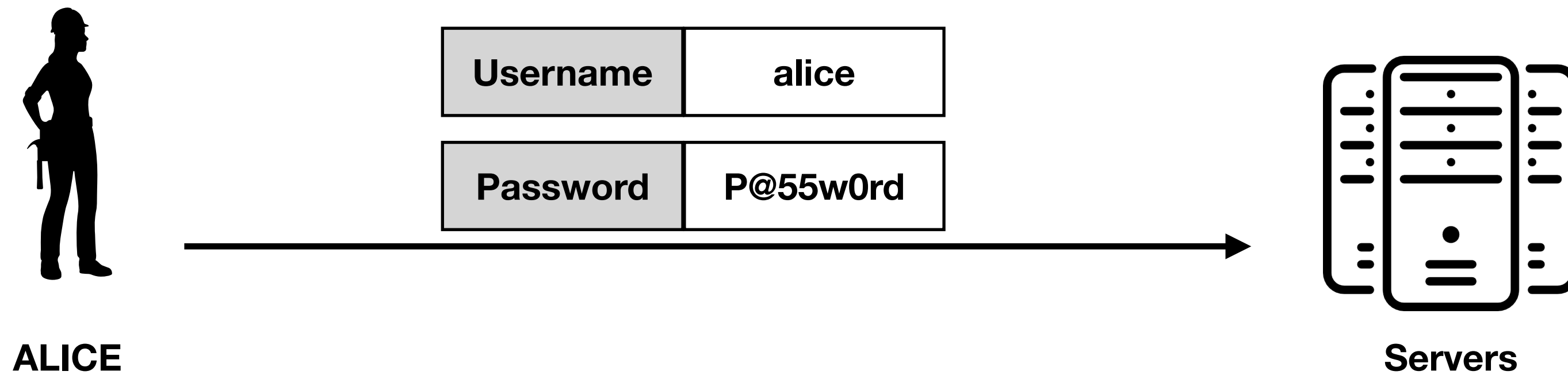
A simple way to store password is by using a database that maps a username with a password. The security strength and resilience of this model depends on **how** the password is stored. The most basic, but also the least secure, password storage format is **cleartext**.

Storing passwords in cleartext is the equivalent of writing them down in a piece of digital paper. Anyone who has access to the database could abuse that access power, retrieve the cleartext credentials, and access the account.

To store password securely we can use cryptographic algorithms. However, they are not all suitable.

Why not use encryption to store passwords?

Password Security



A simple way to store password is by using a database that maps a username with a password. The security strength and resilience of this model depends on **how** the password is stored. The most basic, but also the least secure, password storage format is **cleartext**.

Storing passwords in cleartext is the equivalent of writing them down in a piece of digital paper. Anyone who has access to the database could abuse that access power, retrieve the cleartext credentials, and access the account.

To store password securely we can use cryptographic algorithms. However, they are not all suitable.

Why not use encryption to store passwords?

If we use encryption (i.e., symmetric or asymmetric) that means our password are **reversible**, they can be retrieved using the right key.

Password Security

[Home](#) [Notify me](#) [Domain search](#) [Who's been pwned](#) [Passwords](#) [API](#) [About](#) [Donate](#) 

'--have i been pwned?

Check if your email address is in a data breach

Using Have I Been Pwned is subject to [the terms of use](#)

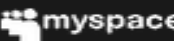
710
pwned websites

12,724,063,603
pwned accounts

115,761
pastes

228,879,575
paste accounts

Largest breaches

	772,904,991 Collection #1 accounts
	763,117,241 Verifications.io accounts
	711,477,622 Onliner Spambot accounts
	622,161,052 Data Enrichment Exposure From PDL Customer accounts
	593,427,119 Exploit.In accounts
	509,458,528 Facebook accounts
	457,962,538 Anti Public Combo List accounts
	393,430,309 River City Media Spam List accounts
	359,420,698 MySpace accounts
	268,765,495 Wattpad accounts

Recently added breaches

	27,917,714 Bureau van Dijk accounts
	1,410,764 PaySystem.tech accounts
	13,204,029 Hjedd accounts
	16,006 Activision accounts
	27,786 Horse Isle accounts
	23,246,481 ApexSMS accounts
	363,468 dBforums accounts
	4,328,232 MalindoAir accounts
	932,232 Viva Air accounts
	836,120 Dymocks accounts

Password Security

Hashing is a common algorithmic way of storing passwords securely.

Why hashing?

Hashing is one way function that generates a representation of the password. Its irreversible. Hashed passwords cannot be un-hashed.

Hash functions used in cryptography have the following key properties:

- It's easy and practical to compute the hash, but "difficult or impossible to re-generate the original input if only the hash value is known."
- It's difficult to create an initial input that would match a specific desired output.

Password Security

Hashing is a common algorithmic way of storing passwords securely.

Why hashing?

Hashing is one way function that generates a representation of the password. Its irreversible. Hashed passwords cannot be un-hashed.

Hash functions used in cryptography have the following key properties:

- It's easy and practical to compute the hash, but "difficult or impossible to re-generate the original input if only the hash value is known."
- It's difficult to create an initial input that would match a specific desired output.

Hashing uses mathematical operations that are extremely difficult and cumbersome to revert such as the modulo operator.

$$5 \bmod 3 = 2$$

This operation is deterministic, given the same input always produces the same output, i.e., 2.

But again,

$$7 \bmod 5 = 2$$

By storing 2, we could never know exactly which numbers were used to generate 2.

This is an important characteristic of a modulo operation, is we cannot find the original operands given the result.

Password Security

Hashing is a common algorithmic way of storing passwords securely.

Why hashing?

The irreversible mathematical properties of hashing make it a phenomenal mechanism to conceal passwords at rest and in motion.

Another critical property that makes hash functions suitable for password storage is that they are deterministic.

A deterministic function is a function that given the same input always produces the same output. This is vital for authentication since we need to have the guarantee that a given password will always produce the same hash; otherwise, it would be impossible to consistently verify user credentials with this technique.

Password Security

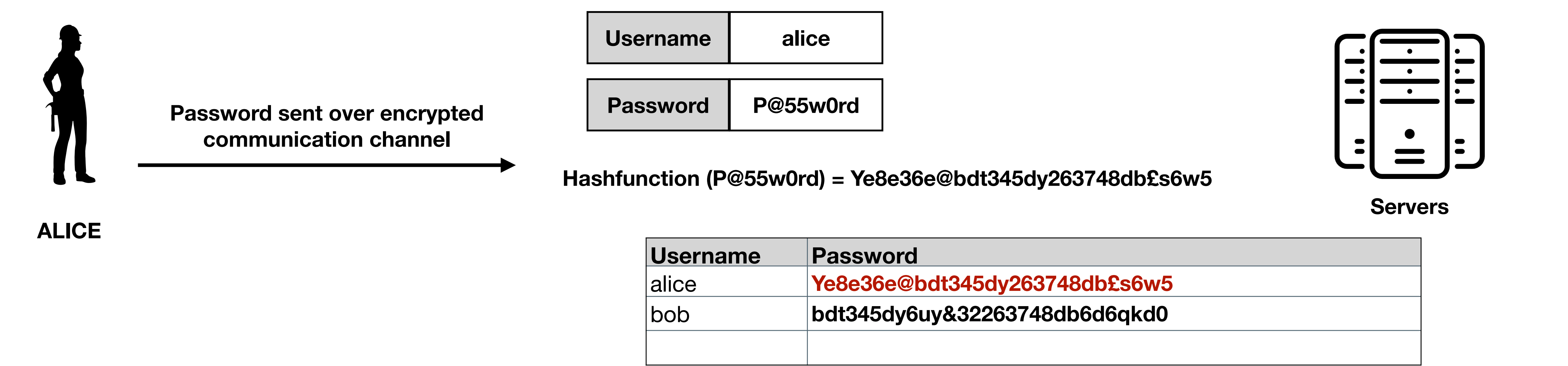
Hashing is a common algorithmic way of storing passwords securely.

Why hashing?

The irreversible mathematical properties of hashing make it a phenomenal mechanism to conceal passwords at rest and in motion.

Another critical property that makes hash functions suitable for password storage is that they are deterministic.

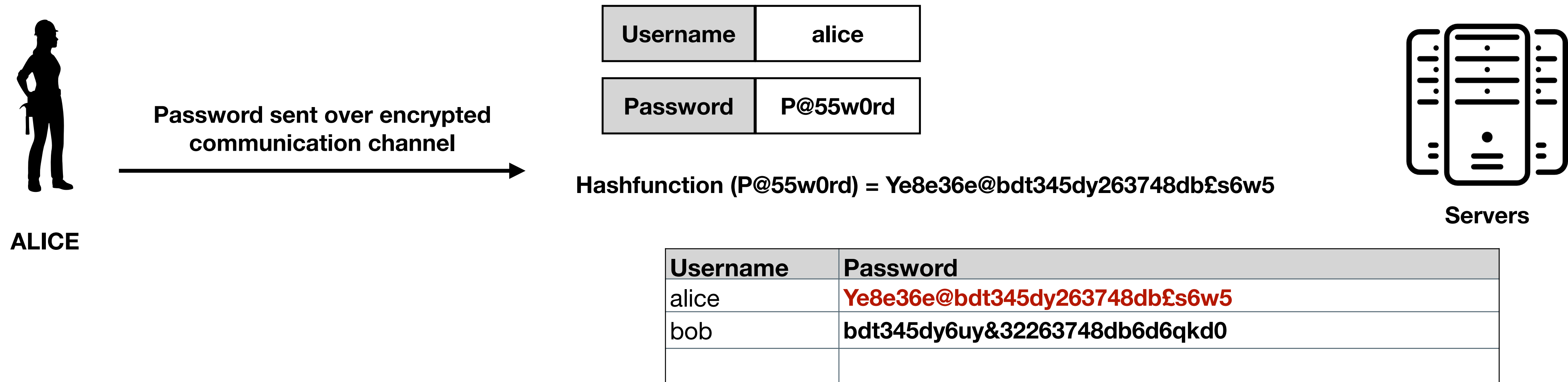
A deterministic function is a function that given the same input always produces the same output. This is vital for authentication since we need to have the guarantee that a given password will always produce the same hash; otherwise, it would be impossible to consistently verify user credentials with this technique.



Password Security

Limitations of using hashing for passwords

A weak password can be exploited. An attacker can use several attacks to search for the right password: dictionary attacks, rainbow table attack, or brute force. Memory, processing power and bandwidth are cheap these days.



Password Security

Hashing - SALT

Additional steps can be taken to protect the contents of the digests. These may include using salts, pepperering, and key stretching.

Salting is where you add an extra bit of data to the password before you hash it while **key stretching** is using a specialised password hash algorithm that is intentionally designed to be slower. **Pepperering** strategies is another layer of protection, it involves hashing the passwords as usual (using a password hashing algorithm) and then HMAC or encrypting the hashes with a symmetrical encryption key before storing the password hash in the database, with the key acting as the pepper.

Password Security

Hashing - SALT

Additional steps can be taken to protect the contents of the digests. These may include using salts, pepperering, and key stretching.

Salting is where you add an extra bit of data to the password before you hash it while **key stretching** is using a specialised password hash algorithm that is intentionally designed to be slower. **Pepperering** strategies is another layer of protection, it involves hashing the passwords as usual (using a password hashing algorithm) and then HMAC or encrypting the hashes with a symmetrical encryption key before storing the password hash in the database, with the key acting as the pepper.

Hashfunction (P@55w0rd) = Ye8e36e@bdt345dy263748db£s6w5

Salting = Hashfunction (Salt, Password)

Hashfunction (Salt, P@55w0rd) = 8e16e@bt3y263748db£s6w0e0r*4f4\$5

Jeff Atwood, "hashes, when used for security, need to be slow."

Salts make dictionary attacks and brute force attacks for cracking large number of passwords much more difficult and limit the impact of rainbow tables. By salting a password, we make it very expensive for an attacker to guess a password.

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Password Security

Common password hashing algorithms

- **SHA** (secure hashing algorithm) family:
 - **SHA-256**: This hashing algorithm is a variant of the SHA2 hashing algorithm, recommended and approved by the NIST. It generates a 256-bit hash value.
 - **SHA-384**: This hashing algorithm is the latest member of the SHA family, it's much faster than the SHA-256 and it's based on a totally different approach (sponge construction).
- **Whirlpool**: This hashing algorithm is based on the advanced encryption standard (AES) and produces a 512-bit hash digest.
- **RIPMED** (RACE integrity primitives evaluation message digest): This hashing algorithm is based on the old MD4 algorithm. The latest and more secured version is RIPMED-320.
- **CRC32** (cyclic redundancy check): This hashing algorithm is used to identify networks errors and disk write errors in networks and storage devices. It focuses on speed instead than security.
- **Argon2**: This hashing algorithm specifically designed to protect credentials. Considered one of the most secure and recommended by the Open Web Application Security Project ([OWASP](#)), it provides a high level of defense against GPU-based attacks.
- **Bcrypt**: This password hashing function was built to slow down brute force attacks. Salt is included in the hashing process, which protects your stored hash values against rainbow tables attacks.
- **PBKDF2** (password-based key derivation function): Recommended by NIST, this hashing algorithm is much slower than SHA, therefore, it's another suitable hashing algorithm for password protection. It uses a salt to generate harder-to-guess password hashes and it produces an output of configurable size (e.g., 256 bits).

Further Reading

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html